

SIC - 2026

Banking System Project

1. Problem Statement

Traditional banking systems are complex, highly secure, and not easy to understand for beginners learning software development. There is a need for a simplified banking simulation system that demonstrates core banking operations such as account management, transactions, and analytics in a structured and interactive way.

This project aims to design a lightweight banking system using Python and Streamlit that simulates real-world banking functionality while remaining easy to understand, extend, and demonstrate.

2. Project Objectives

The main objectives of this project are:

- To simulate basic banking operations such as account creation, deposit, withdrawal, and fund transfer
- To implement secure user authentication using simple password-based login
- To maintain persistent storage of accounts and transactions using JSON and CSV files
- To implement undo functionality for reversing recent transactions
- To provide a user-friendly web interface using Streamlit
- To generate meaningful analytics and visual reports using Matplotlib
- To separate system logic into modular components for better maintainability
- To demonstrate object-oriented programming concepts in a real-world scenario

3. Files Used

app.py - Main Streamlit application containing business logic, data handling, analytics, and user interface.

accounts.json - Stores account details, balances, passwords, and transaction history.

transactions.csv - Stores transaction logs in tabular form.

4. Main Technologies

Python

Streamlit

JSON file storage

CSV transaction logging

Matplotlib for charts

5. Main Classes

DataManager

Handles loading and saving account data from JSON and logging transactions to CSV.

Account

Represents a bank account. It stores account number, name, password, balance, transaction history, and undo stack.

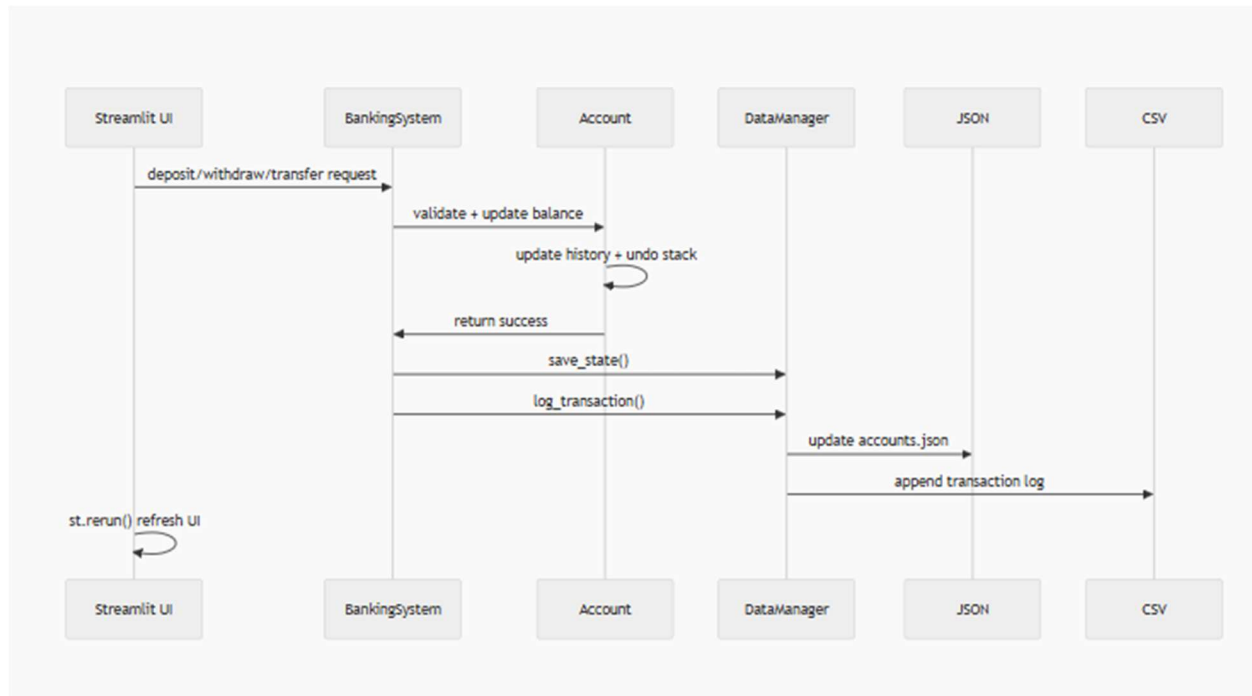
BankingSystem

Controls account creation, login, transfer, data loading, data saving, and bank-level statistics.

AnalyticsManager

Generates charts for monthly transactions, balance trends, top customers, and transaction counts

Execution Flow:



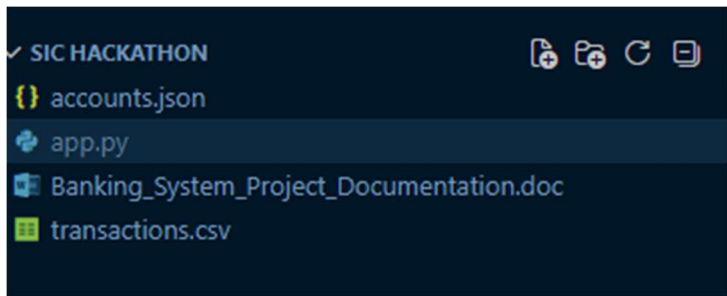
6. Data Storage Implementation

The application stores account data in `accounts.json` and transaction logs in `transactions.csv`. JSON is used for structured account records, while CSV is used for transaction reporting.

```
class DataManager:
    JSON_FILE = "accounts.json"
    CSV_FILE = "transactions.csv"

    @classmethod
    def load_accounts(cls):
        if os.path.exists(cls.JSON_FILE):
            with open(cls.JSON_FILE, "r") as f:
                data = json.load(f)
                if data:
                    return data
        return

    @classmethod
    def save_accounts(cls, accounts_dict):
        with open(cls.JSON_FILE, "w") as f:
            json.dump(accounts_dict, f, indent=4)
```

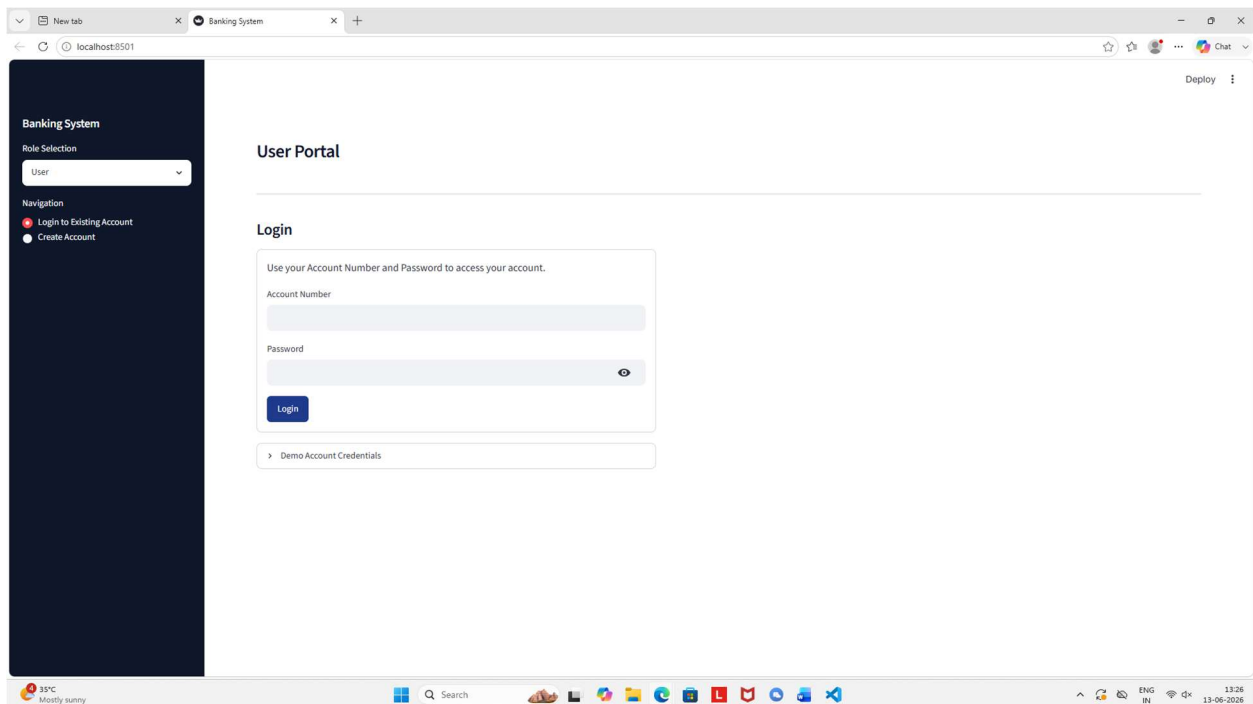


7. Account Creation Implementation

The create_account function validates the name, password, and opening balance. It then generates the next account number, creates an Account object, stores it in memory, and saves the updated data to accounts.json.

```
def create_account(self, name, password, balance):
    if not name or not name.strip():
        return False, "Account holder name cannot be empty.", None
    if not password:
        return False, "Password cannot be empty.", None
    if balance < 0:
        return False, "Opening balance cannot be negative.", None

    acc_num = self.generate_next_account_number()
    undo_stack_ref = self.session_state.undo_stacks[acc_num]
    new_account = Account(acc_num, name.strip(), password, balance, [], undo_stack_ref)
    self.accounts[acc_num] = new_account
    self.save_state()
    return True, "Account created successfully.", acc_num
```



8. Login Implementation

The login function checks whether the entered account number exists and whether the entered password matches the saved password.

```
def login(self, account_number, password):
    account = self.get_account(account_number)
    if account and account.password == password:
        return True
    return False
```

User Portal

Login

Use your Account Number and Password to access your account.

Account Number

Password



Login

> Demo Account Credentials

9. Deposit and Withdraw Implementation

Deposit increases the balance and withdraw decreases the balance. Both functions validate the amount, create a transaction record, add it to history, and push it to the undo stack.

```
def deposit(self, amount, timestamp=None):
    if amount <= 0:
        raise ValueError("Deposit amount must be greater than zero.")
    self.balance += amount
    ts = timestamp or datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    txn = {"type": "deposit", "amount": amount, "timestamp": ts}
    self.add_history(txn)
    self.push_undo(txn)

def withdraw(self, amount, timestamp=None):
    if amount <= 0:
        raise ValueError("Withdrawal amount must be greater than zero.")
    if self.balance < amount:
        raise ValueError("Insufficient balance.")
    self.balance -= amount
    ts = timestamp or datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    txn = {"type": "withdraw", "amount": amount, "timestamp": ts}
    self.add_history(txn)
    self.push_undo(txn)
```

Welcome, Rudraksh

Account Holder Name
Rudraksh

Account Number
1012

Current Balance
₹10,000,001

Total Transactions
4

Deposit Funds

Current Balance

₹10,000,001

Amount (₹)

100000.00 - +

Submit Deposit

10. Transfer Implementation

The transfer function moves money from one account to another. It validates both accounts, checks balance, records the sender transaction, records the receiver transaction, saves the state, and logs both transactions in CSV.

```
def transfer(self, sender_acc_num, receiver_acc_num, amount):
    sender = self.get_account(sender_acc_num)
    receiver = self.get_account(receiver_acc_num)

    if not sender or not receiver:
        return False, "Account not found."
    if sender_acc_num == receiver_acc_num:
        return False, "Sender and receiver cannot be the same account."
    if amount <= 0:
        return False, "Transfer amount must be greater than zero."
    if sender.balance < amount:
        return False, "Insufficient balance in sender's account."

    sender.balance -= amount
    receiver.balance += amount
    sender_txn = "type": "transfer", "to": receiver_acc_num, "amount": amount
    receiver_txn = "type": "received_transfer", "from": sender_acc_num, "amount": amount
    sender.add_history(sender_txn)
    sender.push_undo(sender_txn)
    receiver.add_history(receiver_txn)
    self.save_state()
    return True, "Transfer completed successfully."
```

Transfer Funds

Current Balance

₹11,400,001

Recipient Account Number

1005

Amount (₹)

500000.00 - +

✓ Recipient Verified. Account Holder: Ethan Miller

Transfer Details Review

From Account: 1012

To Account: 1005

Recipient: Ethan Miller

Amount: ₹500,000

Confirm & Execute Transfer

11. Undo Stack Implementation

The undo stack is a list that stores reversible transactions. Whenever a user performs a deposit, withdraw, or transfer, the transaction is pushed into the stack. The latest transaction can then be removed using pop and reversed. This follows the Last In First Out method.

```
def push_undo(self, txn):
    self.undo_stack.append(txn)

def pop_undo(self):
    if not self.undo_stack:
        return None
    return self.undo_stack.pop()
```

Welcome, Rudraksh 

Account Holder Name
Rudraksh

Account Number
1012

Current Balance
₹11,400,001

Total Transactions
20

Undo Transaction

Last Undoable Transaction

Type: Deposit

Amount: ₹100,000

Date: 2026-06-13 13:28:05

Undo Transaction

12. Undo Functionality Implementation

The `undo_last_transaction` function checks the last reversible transaction. If it was a deposit, the amount is subtracted. If it was a withdrawal, the amount is added back. If it was a transfer, the receiver balance is reduced and the sender balance is restored. Undo actions are also added to transaction history and CSV logs.

```
def undo_last_transaction(self, banking_system):
    last_txn = self.pop_undo()
    if not last_txn:
        return False, "No transactions to undo."

    if last_txn["type"] == "deposit":
        self.balance -= last_txn["amount"]
        self.add_history("type": "undo_deposit", "amount": last_txn["amount"])
        return True, "Deposit undone."

    elif last_txn["type"] == "withdraw":
        self.balance += last_txn["amount"]
        self.add_history("type": "undo_withdraw", "amount": last_txn["amount"])
        return True, "Withdrawal undone."

    elif last_txn["type"] == "transfer":
        receiver = banking_system.get_account(last_txn["to"])
        amount = last_txn["amount"]
        if receiver.balance < amount:
            self.push_undo(last_txn)
            return False, "Cannot undo transfer."
        receiver.balance -= amount
        self.balance += amount
        self.add_history("type": "undo_transfer", "amount": amount)
        receiver.add_history("type": "undo_received_transfer", "amount": amount)
        return True, "Transfer undone."
```

Welcome, Rudraksh

Account Holder Name
Rudraksh

Account Number
1012

Current Balance
₹11,400,001

Total Transactions
20

Transaction History

Date	Transaction Type	Amount	Related Account	Description
2026-06-13 13:29:02	Transfer Out Reversal	₹500,000	1005	Reversed transfer to Account 1005
2026-06-13 13:28:58	Transfer Out	₹500,000	1005	Transferred to Account 1005
2026-06-13 13:28:05	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:03	Deposit	₹100,000	-	Cash Deposit

13. Transaction History Implementation

Each account stores transactions inside the history list. Streamlit displays this list as a dataframe for the logged-in user.

```
elif menu == "Transaction History":
    st.markdown("#### Transaction History")
    if not account.history:
        st.info("No transactions yet for this account.")
    else:
        st.dataframe(account.history)
```

Welcome, Rudraksh

Account Holder Name
Rudraksh

Account Number
1012

Current Balance
₹11,400,001

Total Transactions
20

Transaction History

Date	Transaction Type	Amount	Related Account	Description
2026-06-13 13:29:02	Transfer Out Reversal	₹500,000	1005	Reversed transfer to Account 1005
2026-06-13 13:28:58	Transfer Out	₹500,000	1005	Transferred to Account 1005
2026-06-13 13:28:05	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:04	Deposit	₹100,000	-	Cash Deposit
2026-06-13 13:28:03	Deposit	₹100,000	-	Cash Deposit

14. Analytics Implementation

The analytics module uses transaction history and account balances to generate visual charts. It shows monthly transactions, balance trends, top customers, and transaction counts.

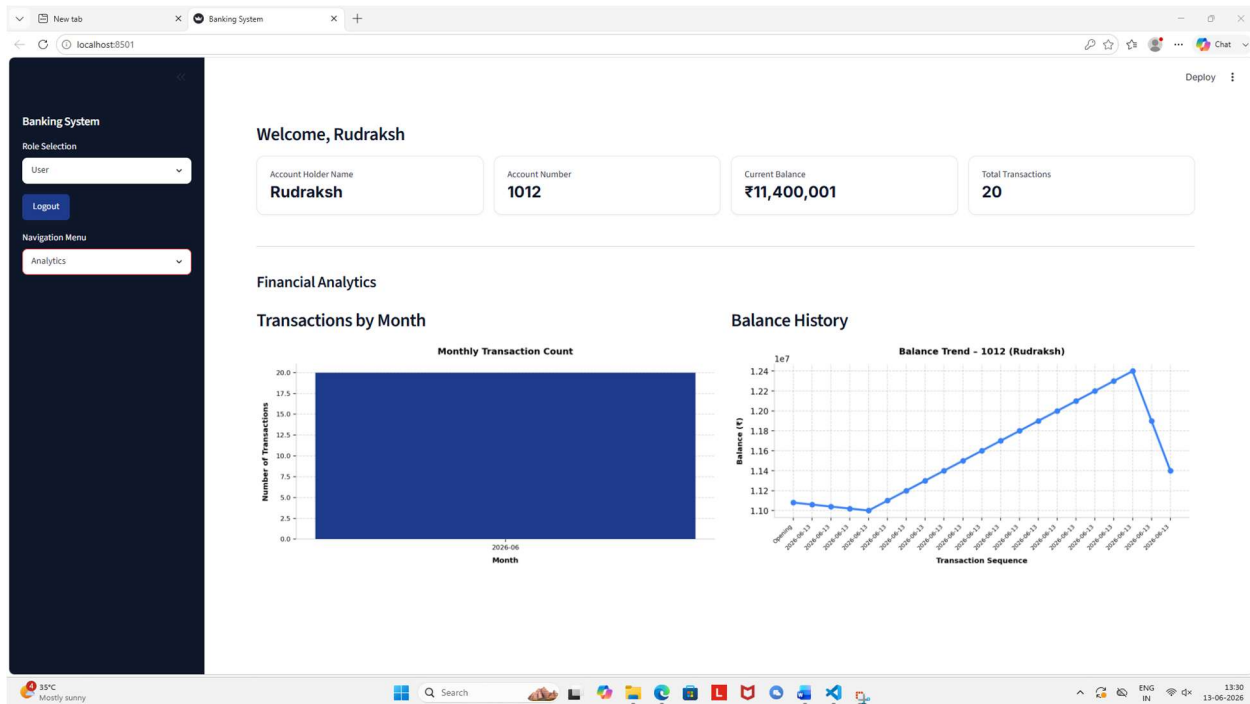
```
def monthly_transactions(account):
    months = []
    for txn in account.history:
        dt = datetime.strptime(txn["timestamp"], "%Y-%m-%d %H:%M:%S")
        months.append(dt.strftime("%Y-%m"))

    month_counts = Counter(months)
    sorted_months = sorted(month_counts.keys())
```



```
counts = [month_counts[m] for m in sorted_months]
```

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(sorted_months, counts)
st.pyplot(fig)
```



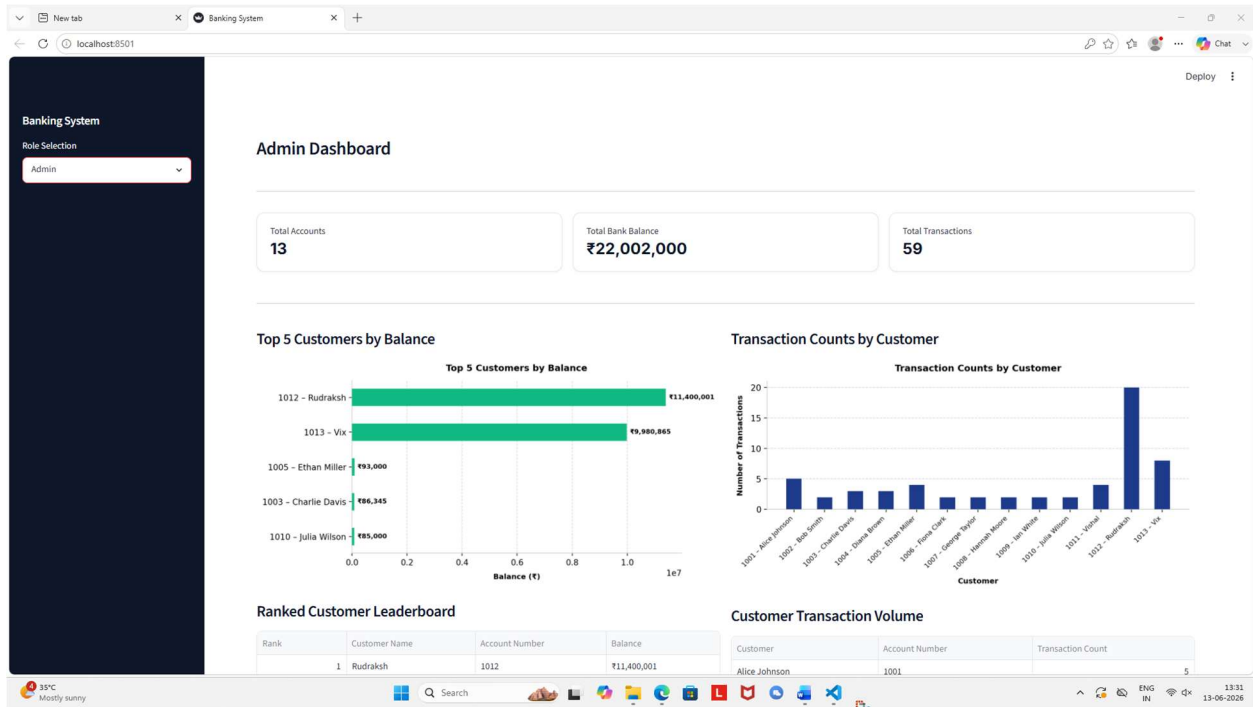
15. Admin Dashboard Implementation

The admin dashboard gives an overview of the complete banking system. It shows total accounts, total bank balance, and total transactions. The admin does not need to login with an account because this section is used for monitoring the system.

```
if role == "Admin":
    st.session_state.logged_in_account = None
    st.markdown("### Admin Dashboard")

    if not banking_system.get_all_accounts():
        st.warning("No accounts in the system.")
        return

    with st.container():
        st.markdown("#### Overall Bank Summary")
        col1, col2, col3 = st.columns(3)
        col1.metric("Total Accounts", len(banking_system.get_all_accounts()))
        col2.metric("Total Bank Balance", f"Rs. {banking_system.get_total_balance():.0f}")
        col3.metric("Total Transactions", banking_system.get_total_transactions())
```



16. Admin Summary Functions

The admin dashboard uses helper functions from BankingSystem. These functions calculate total balance and total transactions from all available accounts.

```
def get_total_balance(self):  
    return sum(acc.balance for acc in self.accounts.values())  
  
def get_total_transactions(self):  
    return sum(len(acc.history) for acc in self.accounts.values())
```

Admin Dashboard



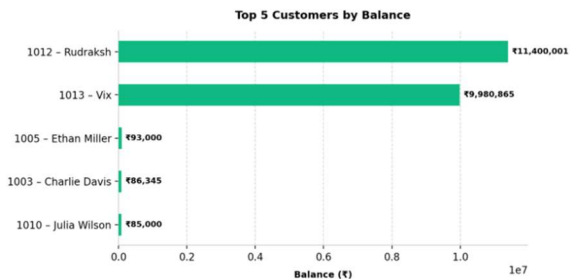
17. Admin Top Customers Analytics

The admin can view the top five customers based on account balance. The accounts are sorted in descending order and displayed using a horizontal bar chart.

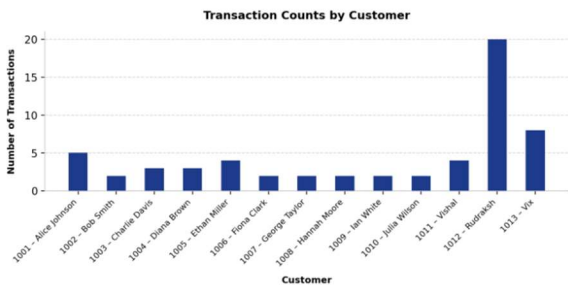
```
def get_top_customers(self):
    sorted_accs = sorted(self.accounts.values(), key=lambda acc: acc.balance, reverse=True)[:5]
    return [(f"acc.account_number - acc.name", acc.balance) for acc in sorted_accs]

def top_customers(banking_system):
    top = banking_system.get_top_customers()
    names, balances = zip(*top)
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.barh(names, balances)
    ax.invert_yaxis()
    st.pyplot(fig)
```

Top 5 Customers by Balance



Transaction Counts by Customer



Ranked Customer Leaderboard

Rank	Customer Name	Account Number	Balance
1	Rudraksh	1012	₹11,400,001
2	Vix	1013	₹9,980,865
3	Ethan Miller	1005	₹93,000
4	Charlie Davis	1003	₹86,345
5	Julia Wilson	1010	₹85,000

Customer Transaction Volume

Customer	Account Number	Transaction Count
Alice Johnson	1001	5
Bob Smith	1002	2
Charlie Davis	1003	3
Diana Brown	1004	3
Ethan Miller	1005	4
Fiona Clark	1006	2
George Taylor	1007	2
Hannah Moore	1008	2
Ian White	1009	2
Julia Wilson	1010	2

18. Admin Transaction Count Analytics

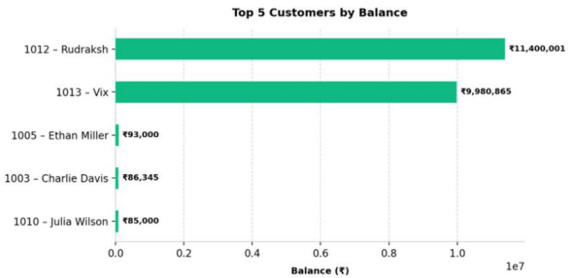
The admin can also view how many transactions each customer has performed. This helps compare account activity across customers.

```
def get_transaction_counts(self):
    acc_labels = [f"acc.account_number - acc.name" for acc in self.accounts.values()]
    txn_counts = [len(acc.history) for acc in self.accounts.values()]
    return acc_labels, txn_counts

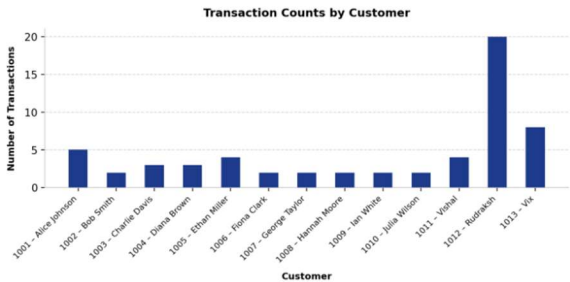
def transaction_counts(banking_system):
```

```
acc_labels, txn_counts = banking_system.get_transaction_counts()
fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(acc_labels, txn_counts)
ax.set_xlabel("Customer")
ax.set_ylabel("Number of Transactions")
plt.show()
```

Top 5 Customers by Balance



Transaction Counts by Customer



Ranked Customer Leaderboard

Rank	Customer Name	Account Number	Balance
1	Rudraksh	1012	₹11,400,001
2	Vix	1013	₹9,980,865
3	Ethan Miller	1005	₹93,000
4	Charlie Davis	1003	₹86,345
5	Julia Wilson	1010	₹85,000

Customer Transaction Volume

Customer	Account Number	Transaction Count
Alice Johnson	1001	5
Bob Smith	1002	2
Charlie Davis	1003	3
Diana Brown	1004	3
Ethan Miller	1005	4
Fiona Clark	1006	2
George Taylor	1007	2
Hannah Moore	1008	2
Ian White	1009	2
Julia Wilson	1010	2

19. Conclusion

The project demonstrates basic banking operations using Python and Streamlit. It includes persistent account storage, transaction logging, undo functionality, user and admin views, and analytics. The structure is simple and suitable for demonstration, learning, and hackathon presentation.